# Ders 13 Genişletilmiş Notu: Sanal Bellek: Adres Dönüşümü, Sayfa Tablosu, TLB ve Sayfa Yerine Koyma

## Ders 13 Genişletilmiş Notu: Sanal Bellek: Adres Dönüşümü, Sayfa Tablosu, TLB ve Sayfa Yerine Koyma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Dersin Çerçevesi
- 2. Önbellek Tekrarı
- 3. Bu Bölümün Yol Haritası
- 4. Genelleştirilmiş Önbellekleme ve Ortalama Erişim Süresi
- 5. Bellek Hiyerarşisinde Sanal Belleğin Yeri
- 6. Sanal Belleğin Temel Kavramları
- 7. Sayfa Tablosu ile Adres Dönüşümü
- 8. Adres Dönüşümünün Genel Tanımı ve Sayfa Hatası
- 9. Önbellek ile Sanal Belleğin Karşılaştırılması
- 10. TLB: Dönüşümü Hızlandıran Önbellek
- 11. TLB ile Adres Dönüşümünün Akışı
- 12. Örnek 1: 48 Bit Sanal Adres ve Sayfa Tablosu Büyüklüğü
- 13. Örnek 2: Fully Associative Küçük TLB
- 14. Sayfa Yerine Koyma: FIFO ve LRU
- 15. Örnek 3: 2-Way Set Associative, 512 Girdili TLB
- 16. Örnek 4: TLB ve Sayfa Tablosu Birlikte


## 1. Dersin Çerçevesi

Bu derste sanal bellek (virtual memory) konusuna girildi. Hoca, konuyu olabildiğince örneklerle anlatacağını söyledi. Konu İşletim Sistemleri dersiyle de bağlantılıdır. Donanım altyapısını bilen biri, bu kavramları o derste daha kolay birleştirir.

> **Hocanın vurgusu:** Son haftalar, mesleğiniz ve öğrenmeniz gereken kavramlar açısından en önemli haftalardır.

Bu dersi anlamanın ön koşulu önbelleği (cache memory) iyi kavramış olmaktır: nasıl kullanıldığı, neden gerektiği ve hit rate'in neden bu kadar yüksek olduğu. Bunlar oturduysa sanal bellek, üstüne eklenen yeni bir kavram olarak kolayca yerleşir.

## 2. Önbellek Tekrarı

### 2.1 Önbellek Neden Var?

Son iki haftada işlenen önbellek kısaca hatırlatıldı. Önbellek, kullanılması zorunlu bir yapı değildir. Bir bilgisayarda olmazsa olmaz birimler işlemci ve bellektir. Aritmetik işlemleri işlemci yapar. Bellek gerekir, çünkü verinin tamamını register'larda tutamayız.

Önbelleğin amacı işlemleri hızlandırmaktır. Veriyi ana bellekten, yani dinamik RAM'den register'a taşımak erişim süresi açısından çok yavaştır. Bu yüzden ana bellek ile işlemci arasına bir önbellek yerleştirilir. Önbellek, ana bellekteki verinin küçük bir parçasını, sık erişilen bir bölgeyi tutar. İşlemci bu veriye daha hızlı erişir.

**Soru:** Ana bellekten önbelleğe veri alma politikasını işletim sistemi tarafında mı belirliyoruz, yoksa bunun donanımsal bir karşılığı mı var?

**Cevap:** Donanımla belirlenir. Daha önce görülen eşleme tekniklerinden (fully associative, direct mapped gibi) hangisi tasarlanıp üretildiyse o kullanılır. İşlemcinin özelliklerinde örneğin "4-way set associative, şu kadar kapasite" yazar.

**Soru:** Kastım, tek tek hangi verinin oraya kaydedileceği. Sık kullanılanlar demiştik.

**Cevap:** Ana bellekte bir adrese erişildiğinde, o adresin çevresindeki blok önbelleğe taşınır.

> **Ders sonrası not:** Hoca bu taşımayı işletim sisteminin yaptığını söyledi; doğrusu büyük olasılıkla önbellek bloğunun yüklenmesini donanımın (önbellek denetleyicisinin) yapmasıdır. İşletim sistemi büyük olasılıkla sayfa düzeyinde, yani sanal bellek tarafında devreye girer.

**Soru:** O zaman işletim sistemini aşıp istediğimiz bazı verileri kendimiz önbelleğe kaydedip oradan işlem yapabilir miyiz?

**Cevap:** Özel bir tasarım kullanırsanız yapabilirsiniz. Ama ihtiyacı öngörme konusunda orada zaten iyi bir algoritma çalışıyor.

### 2.2 Her Seviye Bir Sonrakinin Alt Kümesidir

Hocanın "çok güzel" bulduğu cümle şuydu: *Each level of memory hierarchy is just a subset of next higher level.* Bellek hiyerarşisinde her seviye, bir sonraki daha büyük seviyedeki belleğin bir alt kümesini saklar.

Önbellek, ana bellekteki küçük bir adres bölgesini tutar. Aynı ilişki disk ile ana bellek arasında da vardır. Diskteki verinin tamamı ana belleğe taşınmaz. Yalnız erişilmek istenen dosya ya da çalıştırılacak program adreslendiğinde diskten ana belleğe gelir.

> **Ders sonrası not:** Hoca "üst seviye" (upper/higher level) sözünü iki anlamda kullandı. Bu cümlede ve sanal bellek için "higher level" daha büyük kapasiteli seviyeyi anlatır. Piramit anlatımında ise "upper level" CPU'ya yakın, küçük ve hızlı seviyedir. Bu yüzden her kullanımda hangi yönün kastedildiğine bakın.

### 2.3 Zamansal ve Uzamsal Yerellik

Önbellek bu hızlandırmayı iki özellik sayesinde başarır. Hit rate'in yüksek olmasının sebebi bunlardır.

- **Zamansal yerellik (temporal locality):** Referans verilen bir adrese yakın gelecekte tekrar referans verilebilir. Eriştiğiniz bir adresi 3-5 satır kod sonra yeniden çağırma ihtimaliniz çok yüksektir. Bu öngörüldüğü için erişilen adres önbelleğe taşınır.
- **Uzamsal yerellik (spatial locality):** Erişilen adrese komşu olan, onun bir üstündeki ya da altındaki satırlara da erişilmesi beklenir. Bu yüzden komşu bölge blok olarak önbelleğe taşınır. Blok boyutunu bir word'den büyük seçmek uzamsal yerelliği destekler ve aranan veriyi önbellekte bulma şansını artırır.

  > **Hocanın vurgusu:** Bu ders bittikten 5-10 yıl sonra bile aklınızda kalması gereken kavramlar zamansal ve uzamsal yerelliktir. Yüksek lisans mülakatlarında çokça sorulur. Hoca, mülakatlarda sorulan bir başka kavram olarak istatistikten standart sapmayı da andı.

### 2.4 Önbellek Tasarım Seçenekleri

Slaytta önbellek tasarımındaki parametreler sıralandı:

- **Kapasite (size of cache):** Kapasite arttıkça sistem hızlanır. Ancak önbellek statik RAM ile üretildiği için maliyet çok artar, bu yüzden kapasite çok büyütülemez.

- **Eşleme yapısı:** Direct mapped, associative ve set associative yapılar görüldü. N-way set associative yapıda N'in kaç seçileceği de bir tasarım kararıdır.
- **Blok yerine koyma politikası (block replacement policy):** Örneğin 4 setli bir yapıda dört set de doluyken aynı index'e sahip yeni bir adres gelirse hangi sete yerleşeceğine karar verilir. Least recently used ve random gibi algoritmalar vardır. Bunlar İşletim Sistemleri dersinde bolca konuşulacak.
- **İkinci seviye önbellek (second level cache):** Level 1 ve Level 2 kavramları.
- **Yazma politikası:** Write-through ve write-back.

**Soru:** Yeni nesil işlemcilere baktım, onlarda bile 12 megabayt var.

**Cevap:** Level 1, Level 2 gibi seviyeler bu yüzden kullanılıyor. Maliyet burada çok önemli bir unsurdur ve yapı daha fazla büyütülemiyor.

## 3. Bu Bölümün Yol Haritası

Sanal bellek bölümünde işlenecek başlıklar şunlardır:

- **Genelleştirilmiş önbellekleme (generalized caching):** Önbellek fikri başka nerelerde kullanılabilir?
- **Adres dönüşümü (address translation):** Sanal adresten fiziksel adrese dönüşüm.
- **Sayfa tablosu (page table):** Bu dönüşümde kullanılan tablo. Derste genel yapısı verilecek. Tabloyu bölerek kullanma ve hiyerarşik tablolar İşletim Sistemleri dersinde görülecek.
- **TLB (Translation Lookaside Buffer):** Sayfa tablosu çok büyüktür ve ana bellekte tutulur. Bu da sürekli bellek erişimi ve yavaşlık demektir. İşi hızlandırmak için özel bir önbellek tablosu olan TLB kullanılır.
- **Sayfa yerine koyma politikası (page replacement policy):** Önbellekteki yerine koyma politikasının karşılığıdır. Bellek doluysa hangi sayfanın seçileceğini belirler.
- **Çok seviyeli sayfa tabloları (multi-level page tables):** Büyük tabloyu çok seviyeli bölme tekniği. İşletim Sistemleri dersinde işlenecek.

## 4. Genelleştirilmiş Önbellekleme ve Ortalama Erişim Süresi

Önbellek tekniği zamanla bilgisayar sisteminin başka alanlarında da kullanılmıştır: dosya sistemi önbelleği (file system cache), web sayfası önbelleği (web page cache) ve yazılımda memoization. Genel mantığı slayttaki cümle özetler: *If something is expensive but we want to do it repeatedly, just do it once and cache the results.* Pahalı bir işi tekrar tekrar yapacaksanız bir kez yapın ve sonucu saklayın. Önbellekte de böyle oldu: veriyi ana bellekten getirmek yavaş ve maliyetli olduğu için her seferinde bunu yapmak yerine veri bir kez önbelleğe taşındı.

Geçen hafta örnekleri yapılan ortalama erişim süresi şöyle hesaplanır:

$$T_{\text{ortalama}} = \text{hit rate} \times \text{hit time} + \text{miss rate} \times \text{miss time}$$

Burada hit time önbellekten okuma süresidir. Miss time ise önbelleğe bakıp bulamadıktan sonra veriyi ana bellekten getirme süresidir. Hit time, miss time'a göre çok küçük olduğu için sistem performansı bu kadar artırır. Hoca bununla ilgili parametreler olarak önbellek kapasitesini, yerleşim politikasını (layout policy, yani fully associative, direct mapped gibi eşleme teknikleri) ve yerine koyma politikasını saydı.

> **Ders sonrası not:** Hoca bu parametreleri hit time'ın küçük olmasının sebebi olarak ve kapasitenin büyümesini olumlu etken diye andı; doğrusu büyük olasılıkla bu parametrelerin öncelikle hit rate'i belirlemesidir. Daha büyük bir önbellek büyük olasılıkla hit rate'i artırır, ama hit time'ı küçültmez.

## 5. Bellek Hiyerarşisinde Sanal Belleğin Yeri

Hiyerarşi bir piramit olarak düşünülür. Tepesi CPU'ya en yakın bölgedir: CPU içindeki register'lar ve Level 1 önbellek. Altında Level 2 önbellek gelir. Şimdiye kadar ana bellek ile önbellek arasındaki bloklar halinde veri taşınması görüldü.

Dersin kalan kısmı ana bellek ile disk arasındaki veri transferini anlatır. Burada taşıma birimi değişir. Önbellekte **blok** dediğimiz yapının sanal bellek tarafındaki karşılığı **sayfa**dır (page). Sayfalar sabit uzunluklu (fixed size) yapılardır ve diskten belleğe taşınırlar.

Piramidin alt tarafı daha büyük kapasiteli bellek birimleridir. Üst tarafı ise daha küçük kapasiteli ve daha hızlı birimlerdir.

> **Ek bilgi (derste anlatılmadı):** Derste "virtual memory" sözü çoğu yerde doğrudan disk anlamında kullanıldı. Bu kullanım anlatımı kolaylaştırır. Büyük olasılıkla daha kesin ayrım şöyledir: sanal bellek, programın gördüğü adres uzayıdır, disk ise bu uzaydaki sayfaların ana bellekte olmadığında tutulduğu yerdir.

## 6. Sanal Belleğin Temel Kavramları

### 6.1 Dönüşüm İhtiyacı

İşlemci diskteki bir dosyaya ya da programa erişmek istediğinde, sanal adres olarak adresleyebildiği alandaki bu bilgiyi önce fiziksel belleğe (physical memory, yani ana bellek) getirmelidir. Bunun için sanal adresten fiziksel adrese dönüşüm gerekir. Dönüşüm sayfa tablosu üzerinden yapılır.

### 6.2 Yerleştirme, Yerine Koyma ve Swap

Slaytta *size of information blocks that are transferred from secondary to main storage* ifadesi geçer. M harfi ana belleği (main storage) gösterir. Diskten ana belleğe veri, sabit uzunluklu sayfalar halinde taşınır.

- **Yerleştirme politikası (placement policy):** Sayfaların ana belleğe yerleştirilmesi işlemi.
- **Yerine koyma politikası (replacement policy):** Fiziksel belleğin tamamı doluyken yeni bir sayfa gerekiyorsa, dolu gözlerden biri çıkarılır ve yerine yeni sayfa konur.
- **Swap:** Yerine koyma sırasında ana bellekteki sayfa sanal belleğe, sanal bellekteki yeni sayfa da fiziksel belleğe aktarılır.

### 6.3 Donanımsal Eşleme Birimi

Slaytta sol blok *program operates in its virtual address space* der. Program kendi sanal adres uzayında çalışır. Erişilmek istenen sanal adres, *hardware mapping* denen bir bloktan geçer. Bu bloğun görevi sanal adresi fiziksel adrese çevirmektir. Elde edilen fiziksel adresle veri ana bellekten okunur.

### 6.4 Parçalanma (Fragmentation)

Slaytta, sayfaların ana belleğe yerleştirilmesi sırasında karşılaşılan bir sorun gösterilir. Farklı kullanıcılar ya da programlar için (A kullanıcısı, B kullanıcısı, yine A) bellekte ayrı alanlar ayrılır. Bu alanların arasında boşluklar kalır. Bu boşluklar tek başına bir sayfanın yerleşemeyeceği kadar küçüktür, ama bir araya gelince oldukça büyük bir alan oluşturur. Buna parçalanma sorunu denir. Hoca diskte zaman zaman kullanılan birleştirme (defragmentation) uygulamalarını bu boşlukları toparlayan işlemler olarak andı. Konunun ayrıntısı İşletim Sistemleri dersindedir.

> **Ders sonrası not:** Hoca boşlukların bir sayfanın yerleşemeyeceği kadar küçük olduğunu söyledi; doğrusu büyük olasılıkla bu tablonun değişken boyutlu yerleşime (dış parçalanma) ait olmasıdır. Bellek sayfa boyutunda eşit parçalara bölünürse, boş her göze büyük olasılıkla bir sayfa sığar. Sabit sayfalarda büyük olasılıkla asıl kayıp sayfanın içinde kalan boşluktur. Disk birleştirme de büyük olasılıkla ana bellekten ayrı, dosya düzeniyle ilgili bir işlemdir.

## 7. Sayfa Tablosu ile Adres Dönüşümü

### 7.1 32 Sayfalık Slayt Örneği

Hoca "lütfen dikkatli dinleyin" diyerek bu örneğe geçti. Slaytta sağda sanal bellek, solda fiziksel bellek, ortada adres dönüşümü görülür. Değerler kağıt üzerinde gösterilebilsin diye küçük seçilmiştir.

- Sayfa boyutu: 1 KB.
- Sanal bellek: Sayfa 0'dan sayfa 31'e kadar 32 sayfa, toplam 32 KB.
- Fiziksel bellek: 8 KB, yani aynı anda 8 sayfa.

Bu değerlerden adres alanları şöyle çıkar (ara adım eklendi):

$$\text{offset} = \log_2 1024 = 10 \text{ bit}$$

$$\text{sanal adres} = \log_2(32 \times 1024) = 15 \text{ bit}, \quad \text{sanal sayfa numarası} = 15 - 10 = 5 \text{ bit}$$

$$\text{fiziksel adres} = \log_2(8 \times 1024) = 13 \text{ bit}, \quad \text{fiziksel sayfa numarası} = 13 - 10 = 3 \text{ bit}$$

İşlemcide yürütülen komut bu sayfalardan birini, örneğin sayfa 1'deki bir kod parçasını çağırdığında sayfa, fiziksel bellekteki 8 gözden birine yazılır. Hangi sanal sayfanın hangi fiziksel adreste tutulduğunu bilmek için bir tablo gerekir. Sayfa tablosunun görevi budur.

### 7.2 Adresin Bölünmesi

Adres iki alana bölünür:

- **Sayfa içi konum (page offset):** Adresin düşük anlamlı bitleridir. Bit sayısı sayfa boyutundan gelir. 1 KB sayfa için 10 bittir.
- **Sayfa numarası (page number):** Geriye kalan yüksek anlamlı bitlerdir.

Bu bölme hem sanal adres hem de fiziksel adres için yapılır.

### 7.3 Dönüşüm Adımları

1. İşlemciden sanal adres gelir. Düşük anlamlı bitleri offset olarak ayrılır.
2. Yüksek anlamlı kısım, yani sanal sayfa numarası, tabloyu indekslemek için kullanılır. Bu numara örneğin 15 ise tablonun 15. satırına gidilir.
3. Satırdaki valid biti okunur. Bu bit, aranan sayfanın fiziksel belleğe daha önce taşınıp taşınmadığını gösterir. Önbellekte adresin var olup olmadığına bakmaya benzer.
4. Satırda erişim hakları (access rights) da bulunur. Bir sayfaya yalnız okuma ya da okuma ve yazma amacıyla erişilebilir. Bunun için birkaç bit, en az iki bit gerekir.
5. Satırdan fiziksel sayfa numarası (physical page number) okunur.
6. Fiziksel sayfa numarası ile offset yan yana yazılır (concatenate). Sonuç fiziksel adrestir. Slayttaki toplama sembolü aritmetik toplama değil, iki alanı birbirine ekleme anlamındadır.

### 7.4 Sayfa Tablosu Girdisi (PTE)

Tablodaki her satıra sayfa tablosu girdisi (page table entry) denir. Bir girdide üç alan görülür:

- **Valid biti:** Sayfanın fiziksel belleğe getirilip getirilmediğini gösterir.
- **Erişim hakları:** Read only, read-write ya da executable gibi. Ayrıntısı İşletim Sistemleri dersindedir.
- **Fiziksel sayfa numarası.**

  **Ders sonrası not:** Hoca "valid biti 1 ise sayfa virtual memory'de mevcuttur" ve bir yerde "page table'da daha önceden getirilmiş mi" dedi; doğrusu valid 1 iken sayfanın fiziksel bellekte bulunmasıdır. Hocanın aynı anlatımdaki "physical memory'e

taşınmış mı” açıklaması da bunu gösterir. “Physical page table entry” sözü de “page table entry” olmalıdır.

## 8. Adres Dönüşümünün Genel Tanımı ve Sayfa Hatası

Sanal adres uzayı $N$ sayfalı, fiziksel bellek $M$ adresli bir küme olarak verilir. Burada $N > M$, hatta $N \gg M$'dir. Amaç, sanal bellek adresini fiziksel bellek adresine dönüştürmektir (translate).

İşlemci sanal adres uzayındaki bir $a$ adresine erişmek istediğinde adres dönüşüm mekanizması iki sonuçtan birini verir:

- **Fiziksel adres $a'$ döner:** Sayfa ana belleğe daha önce taşınmıştır. Veri doğrudan ana bellekten okunur.
- **0 döner:** Aranan adres fiziksel bellekte yoktur. Buna **sayfa hatası (page fault)** denir. Bu durumda işletim sisteminin page fault handler mekanizması devreye girer ve veriyi diskten ana belleğe aktarır.

## 9. Önbellek ile Sanal Belleğin Karşılaştırılması

Slayttaki karşılaştırma, önbellek ile ana bellek arasındaki ilişkiyi ana bellek ile disk arasındaki ilişkiyle eşleştirir.

| Kavram | Önbellek | Sanal bellek |
|---|---|---|
| Taşınan birim | Blok (line) | Sayfa (page) |
| Bulunamama | Miss | Page fault |
| Birim boyutu | Block size | Page size |
| Yerleştirme | Direct mapped, N-way set associative | Yalnız fully associative (sayfa tablosu üzerinden) |
| Yerine koyma | LRU veya random | Least recently used |
| Yazma | Write-through veya write-back | Yalnız write-back |

Sanal bellekte yalnız write-back kullanılmasının sebebini hoca sayfanın büyük bir yapı olmasıyla açıkladı. Bir kilobaytlık bir sayfanın içindeki bütün güncellemeler önce fiziksel bellekte yapılır. Sayfa swap edileceği ya da üzerine yeni sayfa yazılacağı zaman önce diske yazılır, sonra yerine yeni veri getirilir.

> **Ek bilgi (derste anlatılmadı):** Write-back tercihinin bir başka sebebi büyük olasılıkla disk erişiminin ana belleğe göre çok yavaş olmasıdır. Her yazmada diske gitmek büyük olasılıkla sistemi çok yavaşlatırdı.

> **Ek bilgi (derste anlatılmadı):** Tablodaki “LRU” değeri büyük olasılıkla tam LRU olarak uygulanmaz. Her erişimde sıralama güncellemek pahalıdır, bu yüzden gerçek sistemler büyük olasılıkla referans bitine dayalı yaklaşık LRU yöntemleri kullanır.

**Soru:** Bir önceki slaytta swap ile ilgili bir şey gördüm. Onu anlattınız mı?

**Cevap:** Henüz ayrıntılı anlatılmadı. Örneklerde biraz daha ayrıntılandırılacak.

## 10. TLB: Dönüşümü Hızlandıran Önbellek

Sayfa tablosu ana bellekte tutulduğu için tabloya her erişim yine bir ana bellek erişimi maliyeti getirir. Çözüm, tablonun bir parçasını önbellek yapısında tutmaktır. Slayttaki ifadeyle: *A way to speed up virtual memory translation is to use a special cache of recently used page table entries. This has many names but most frequently used is translation lookaside buffer or TLB.*

Sayfa tablosunda satır seçmek için sanal sayfa numarası kullanılıyordu. TLB ise tablonun yalnız bir kısmını tutar, çünkü hepsi taşınamaz. Bu yüzden önbellekte olduğu gibi TLB'de de sanal adres alanı tutulmalıdır. Böylece sayfa tablosunun hangi satırlarının TLB'ye taşındığı bilinir.

Aranan sanal adres TLB'deki bir değerle eşleşirse **TLB hit** olmuştur ve fiziksel adres, erişim hakları gibi alanlar TLB'den alınır. TLB'de bunların yanında valid biti ve dirty biti de bulunur. Bu alanlar sayfa tablosuna özgüdür.

TLB ana tablonun küçük bir parçasını barındırır. Yine de son zamanlarda sık erişilen adresler buraya taşındığı için hit rate'i oldukça yüksektir. Bu da dönüşümü hızlandırır.

## 11. TLB ile Adres Dönüşümünün Akışı

Slaytta solda işlemci, en sağda içinde sayfa tablosunun da bulunduğu ana bellek vardır. Akış şöyledir:

1. İşlemci sanal adresi önce TLB'de kontrol eder.
2. **TLB hit** ise fiziksel adres okunur. Bu artık bir ana bellek adresidir.
3. Bu adresle önce önbelleğe bakılır, çünkü önbellek ile ana bellek birlikte kullanılmaktadır. Veri önbellekte varsa oradan okunur.
4. Veri önbellekte yoksa (miss) ana bellekten okunur.
5. **TLB miss** ise dönüşüm için ana belleğe, sayfa tablosuna gidilir ve fiziksel adres oradan okunur.

Hoca slayttaki süre varsayımlarını şöyle okudu: önbellekten getirme $\frac{1}{2}T$, ana bellekten getirme $T$, TLB miss sonrası dönüşüm için ana belleğe gitme $20T$. Sisteme gün geçtikçe yeni teknikler eklenmesinin amacı, veriyi bellekten okumayı olabildiğince hızlandırmaktır. Bu sürelerle derste bir toplam hesap yapılmadı.

> **Ders sonrası not:** Hoca ana belleğe gitme süresi için hem $T$ hem $20T$ dedi; aynı bellek için bu iki değer birlikte doğru olamaz. Doğrusu büyük olasılıkla slayttaki etiketlerin TLB bakışı için $\frac{1}{2}T$, önbellek için $T$, ana bellek için $20T$ olmasıdır.

## 12. Örnek 1: 48 Bit Sanal Adres ve Sayfa Tablosu Büyüklüğü

**Verilen:** Sanal bellek sistemi 4 KB sayfa kullanıyor. Word 64 bit. Sanal adres 48 bit. Bir program ve verisi 4395 sayfa gerektiriyor. (a) şıkkı: *What is the minimum number of page tables required?*

**Çözüm (derste yapıldığı gibi):**

1. 1 KB 10 bit ile adreslendiğine göre 4 KB için 12 bit gerekir: $\log_2 4096 = 12$. Page offset 12 bittir.
2. Sanal sayfa numarası geriye kalan kısımdır: $48 - 12 = 36$ bit.
3. Sayfa tablosu sanal sayfa numarasıyla indekslendiği için satırları $0$'dan $2^{36} - 1$'e kadar gider.
4. Tablo $2^{36}$ satırlıktır. Programın kullandığı 4395 sayfanın dönüşümü bu tabloda bulunur.

Hoca soruyu "tablonun kaç satırdan oluşması gerektiği" olarak yorumladı.

> **Ders sonrası not:** Hoca "minimum number of page tables" sorusunu tek tablonun satır sayısı olarak cevapladı; soru metni tablo sayısını sorduğu ve word boyutunu verdiği için istenen büyük olasılıkla çok seviyeli tablo sayısıdır. Hesap şöyledir. Girdi 8 bayt alınırsa tek seviyeli tablo $2^{36} \times 8 = 2^{39}$ bayt, yani 512 GB olur. 4 KB'lık bir tabloya $4096/8 = 512 = 2^9$ girdi sığar. 36 bitlik sanal sayfa numarası $36/9 = 4$ seviyeye bölünür. 4395 sayfa ardışık yerleşirse son seviyede $\lceil 4395/512 \rceil = 9$ tablo, üstteki üç seviyede birer tablo gerekir. Toplam $9 + 1 + 1 + 1 = 12$ tablodur. Çok seviyeli tablolar bu derste değil, İşletim Sistemleri dersinde işlenecektir.

## 13. Örnek 2: Fully Associative Küçük TLB

**Verilen:** *A small TLB has the following entries for a virtual page number of length 20 bits, a physical page number of 12 bits and a page offset of 12 bits.* Sayfa numaraları ve offset onaltılıktır. Her sanal adres için hit olup olmadığı, hit ise fiziksel adres istenir.

Önce adres yapısı çıkarılır:

- Sanal adres: $20 + 12 = 32$ bit.
- Fiziksel adres: $12 + 12 = 24$ bit. Page offset iki tarafta da aynıdır.
- 12 bit, 3 onaltılık hanedir. Offset 3 hane, sanal sayfa numarası 5 hane, fiziksel sayfa numarası 3 hanedir.

TLB içeriği:

| Valid | Dirty | Sanal sayfa no | Fiziksel sayfa no |
|---|---|---|---|
| 1 | 1 | 01AF4 | FFF |
| 0 | 0 | 0E45F | E03 |
| 0 | 0 | 012FF | 2F0 |
| 1 | 0 | 01A37 | 788 |
| 1 | 0 | 02BB4 | 45C |
| 0 | 1 | 03CA0 | 657 |

**(a) Sanal adres 02BB4A65.** Düşük 3 hane offset'tir: A65. Yüksek kısım sanal sayfa numarasıdır: 02BB4. Tabloda bu satır vardır ve valid biti 1'dir. Yani sayfa fiziksel belleğe daha önce taşınmıştır, page fault olmamıştır.

Dirty biti 0'dır. Bu, sayfadaki hiçbir adreste güncelleme yapılmadığı anlamına gelir. Fiziksel bellekteki sayfa ile diskteki sayfa birbiriyle eşleşir, yazma işlemi yapılmamıştır.

Fiziksel sayfa numarası 45C'dir. Arkasına offset eklenir: **fiziksel adres 45CA65**. Bu, $6 \times 4 = 24$ bittir.

**(b) Sanal adres 0E45FB32.** Offset B32, sanal sayfa numarası 0E45F'tir. Bu satır tabloda vardır, fiziksel sayfa numarası da E03'tür.

**Soru:** E03B32 mi çıkıyor?

**Cevap:** Bir şeyi gözden kaçırdınız. Satırın valid biti 0'dır. (Öğrencilerden biri de valid 0 olduğu için fiziksel adres bulunmadığını söyledi.) Hoca bunu page fault olarak yazdı: sayfa diskten fiziksel belleğe taşınır, fiziksel adresi hem sayfa tablosuna hem TLB'ye yazılır.

**(c) Sanal adres 0D34E9DC.** Offset 9DC, sanal sayfa numarası 0D34E'dir. Bu numara tabloda yoktur: **TLB'de yok (not in TLB)**. Bu durumda sayfa tablosuna gidilip bakılır. Sayfa orada olabilir, çünkü TLB'ye taşınmamış olabilir.

**(d) Sanal adres 03CA0777.** Offset 777, sanal sayfa numarası 03CA0'dır. Satır vardır ama valid biti 0'dır. Hoca bunu da page fault olarak yazdı. Dirty biti 1 görünür, ama valid 0 olduğu için erişilmek istenen sayfayla ilişkili değildir.

| Sanal adres | Sanal sayfa no | Offset | Derste verilen sonuç | Fiziksel adres |
|---|---|---|---|---|
| 02BB4A65 | 02BB4 | A65 | Hit | 45CA65 |
| 0E45FB32 | 0E45F | B32 | Page fault (bkz. not) | Yok |
| 0D34E9DC | 0D34E | 9DC | TLB'de yok | Yok |
| 03CA0777 | 03CA0 | 777 | Page fault (bkz. not) | Yok |

**Ders sonrası not:** Hoca (b) ve (d) için "page fault oldu" dedi; doğrusu büyük olasılıkla bunların TLB miss sayılmasıdır. TLB sayfa tablosunun yalnız bir alt kümesidir. Valid 0 olan bir TLB satırı büyük olasılıkla yalnız TLB'de geçerli bir dönüşüm olmadığını gösterir. Page fault olup olmadığını anlamak için hocanın (c) şıkkında söylediği gibi sayfa tablosuna bakmak gerekir. Soru yalnız TLB'yi verdiği için bu tablodan fiziksel adres çıkmaz.

**Ders sonrası not:** Hoca bu örnekte "yüksek anlamlı kısım physical page number" ve "düşük anlamlı 3 bit" dedi; doğrusu aranan alanın virtual page number, ayrılan parçanın da 3 onaltılık hane (12 bit) olmasıdır. Hocanın çözümü bu doğru değerlerle yapıldı.

## 14. Sayfa Yerine Koyma: FIFO ve LRU

Sayfaların fiziksel belleğe taşınmasında random, FIFO, optimal ve least recently used gibi değişik teknikler kullanılır. Slayt örneğinde fiziksel bellek yalnız 3 sayfa tutabilir (*memory system with three pages*) ve başta boştur. Sırayla şu sayfalara erişilir:

$$A\ B\ C\ A\ B\ D\ A\ D\ B\ C\ B$$

Toplam 11 erişim vardır.

### 14.1 FIFO

İlk gelen ilk çıkar. A, B ve C sırayla çerçeve 1, 2 ve 3'e yerleşir. Sonraki A ve B zaten bellektedir. D gelince en eski sayfa A olduğu için D onun üzerine yazılır. A yeniden istenince bu kez B'nin yerine yazılır. D bellektedir. B yoktur ve C'nin yerine yazılır. C yeniden istenince D'nin yerine yazılır.

| Erişim | Çerçeve 1 | Çerçeve 2 | Çerçeve 3 | Sonuç | Çıkan |
|---|---|---|---|---|---|
| A | A | - | - | Page fault | - |
| B | A | B | - | Page fault | - |
| C | A | B | C | Page fault | - |
| A | A | B | C | Hit | - |
| B | A | B | C | Hit | - |
| D | D | B | C | Page fault | A |
| A | D | A | C | Page fault | B |
| D | D | A | C | Hit | - |
| B | D | A | B | Page fault | C |
| C | C | A | B | Page fault | D |
| B | C | A | B | Hit | - |

Sonuç **7 page fault**'tur. Üçü ilk yerleştirme, dördü yerine koymadır. Hoca sayarken ilk yerleştirmelerin sayılıp sayılmadığını düşündü: $3+4=7$ olduğu için slayttaki sayı ilk yerleştirmeleri içerir.

### 14.2 LRU

En uzun süredir kullanılmayan sayfa çıkarılır. Aynı dizide D geldiğinde sayfaların son kullanımına bakılır: A ve B yeni kullanılmıştır, C en eskidir. Bu yüzden FIFO'nun aksine D, C'nin yerine yazılır. Sonraki A, D ve B bellektedir. C geldiğinde en uzun süredir kullanılmayan A çıkar (ara adım eklendi).

| Erişim | Çerçeve 1 | Çerçeve 2 | Çerçeve 3 | Sonuç | Çıkan |
|---|---|---|---|---|---|
| A | A | - | - | Page fault | - |
| B | A | B | - | Page fault | - |
| C | A | B | C | Page fault | - |
| A | A | B | C | Hit | - |
| B | A | B | C | Hit | - |
| D | A | B | D | Page fault | C |
| A | A | B | D | Hit | - |
| D | A | B | D | Hit | - |
| B | A | B | D | Hit | - |
| C | C | B | D | Page fault | A |
| B | C | B | D | Hit | - |

Sonuç **5 page fault**'tur. Aynı dizide LRU, FIFO'dan 2 page fault daha azdır. Her iki sayı da Python ile doğrulandı.

Hoca optimal yöntemin ayrıntısına girmedi. Benzer slayt örneklerini önceki örnek gibi kendiniz çözmeye çalışmanızı istedi.

> **Hocanın vurgusu:** FIFO ve LRU'yu bilmeniz yeterlidir. Diğerlerini İşletim Sistemleri dersinde öğreneceksiniz.
>
> **Ek bilgi (derste anlatılmadı):** Optimal yöntem büyük olasılıkla gelecekte en geç kullanılacak sayfayı çıkarır. Bunun için erişim dizisinin önceden bilinmesi gerekir. Bu tanımla aynı dizinin izi Python ile çıkarıldığında sonuç 5 page fault'tur: D, C'nin yerine yazılır, son C ise A'nın ya da D'nin yerine yazılabilir.

## 15. Örnek 3: 2-Way Set Associative, 512 Girdili TLB

Bu soru önceki yıllara ait, 2010 tarihli bir sorudur. Hoca kavramların aynı kaldığını söyledi.

### 15.1 Verilenler ve Bit Aralıkları

**Verilen:** Sanal adres 32 bit, sanal sayfa numarası (VPN) 24 bit, fiziksel sayfa numarası (PPN) 32 bit. TLB, toplam 512 girdili ve 2-way set associative yapıdadır.

Hoca bunun kötü bir örnek olduğunu belirtti: sanal adres uzayı fiziksel adres uzayından küçük görünüyor. Yine de matematiksel olarak çözülebilir.

1. Page offset: $32 - 24 = 8$ bit.
2. Offset fiziksel adreste de aynıdır, yani 8 bittir.
3. Fiziksel adres: $32 + 8 = 40$ bit.

Soru bu alanları bit aralığı olarak ister:

| Alan | Bit aralığı |
|---|---|
| Virtual page offset (VPO) | Sanal adresin 0-7. bitleri |
| Virtual page number (VPN) | Sanal adresin 8-31. bitleri |
| Physical page offset (PPO) | Fiziksel adresin 0-7. bitleri |
| Physical page number (PPN) | Fiziksel adresin 8-39. bitleri |

### 15.2 TLB Index ve Tag

Önceki örnekteki TLB fully associative idi. Burada ise yapı 2-way set associative olduğu için VPN iki alana bölünmek zorundadır: TLB tag ve TLB index.

512 girdi iki set'e bölünür: $512/2 = 256 = 2^8$. Satırlar 0'dan 255'e kadardır ve bunları indekslemek için 8 bit gerekir. Geriye $24 - 8 = 16$ bit tag kalır.

- **TLB index (TLBI):** VPN'in düşük anlamlı 8 biti, yani 0-7. bitleri.
- **TLB tag (TLBT):** VPN'in 8-23. bitleri, yani 16 bit.

Her set'in bir satırında tag, valid biti, dirty gibi alanlar ve fiziksel sayfa numarası tutulur. TLB önbellek tasarımındaki bütün teknikleri kullanabilir: fully associative, direct mapped ya da set associative olabilir.

> **Ders sonrası not:** Hoca, önbellek dersindeki gibi "iki set, set 0 ve set 1" dedi; doğrusu büyük olasılıkla standart terimlerle index'in seçtiği her satırın bir set, satırdaki iki yerin ise iki way olmasıdır. Hesap değişmez. Bu notta derste kullanılan "set 0" ve "set 1" adları korunmuştur.

### 15.3 TLB İçeriği

Soruda TLB'nin ilgili küçük bir parçası verilir. Index değerleri 8 bit olduğu için 2 onaltılık haneyle gösterilir. Tablonun sol ve sağ tarafı iki ayrı set'tir.

| Index | Valid | Tag | PPN | Valid | Tag | PPN |
|---|---|---|---|---|---|---|
| 3D | 1 | 083F | 0913ABDE | 1 | 083E | AB18ED24 |
| 3E | 0 | F3E9 | 0913ABDE | 1 | 083F | 0118ED24 |
| 3F | 1 | 409A | C013ABDE | 1 | 083E | AC18ED24 |
| 40 | 0 | 083E | 0513ABDE | 1 | 083E | 0918ED24 |

### 15.4 Birinci Adres: 083F3E9A

32 bitlik adres iki alana ayrılır. Düşük 8 bit, yani 2 onaltılık hane offset'tir. Yüksek 6 hane VPN'dir. VPN'in düşük 2 hanesi index, yüksek 4 hanesi tag'dir. Sayılar 4 bitin katı olduğu için binary'e açmaya gerek kalmaz.

| Alan | Değer |
|---|---|
| VPN | 083F3E |
| VPO | 9A |
| TLBI | 3E |
| TLBT | 083F |
| TLB sonucu | Hit |
| PPN | 0118ED24 |
| Fiziksel adres | 0118ED249A |

3E satırındaki iki set de karşılaştırılır. Tag 083F sağ taraftadır ve valid biti 1'dir, yani hit. Fiziksel sayfa numarası 0118ED24'tür. Arkasına offset eklenince 40 bitlik fiziksel adres **0118ED249A** olur.

Hoca çözümde cevap anahtarındaki yanlış bir değeri düzeltti. Kısa nota göre anahtarda AC18ED24 yazıyordu. Tabloya göre doğru değer 3E satırındaki 0118ED24'tür. AC18ED24, 3F satırına aittir.

### 15.5 İkinci Adres: 083E409B

8 onaltılık hane $8 \times 4 = 32$ bit eder.

| Alan | Değer |
|---|---|
| VPN | 083E40 |
| VPO | 9B |
| TLBI | 40 |
| TLBT | 083E |
| TLB sonucu | Hit |
| PPN | 0918ED24 |
| Fiziksel adres | 0918ED249B |

40. satıra gidilir. Sol tarafta 083E vardır, ama valid biti 0'dır. Sağ tarafta da 083E vardır ve valid biti 1'dir. Geçerli olan sağ taraftır, yani hit. Fiziksel sayfa numarası 0918ED24, 40 bitlik fiziksel adres ise **0918ED249B**'dir.

    **Hocanın vurgusu:** Soruda bir numara var. Set associative yapıda bir index altındaki bütün setlere bakın. Sol taraftaki valid 0'ı görüp öbür sete bakmasaydık hata yapacaktık.

## 16. Örnek 4: TLB ve Sayfa Tablosu Birlikte

Bu örneği hoca bir önceki örnekten daha kapsamlı olarak tanıttı. Bu kez hem TLB hem de sayfa tablosu verilmiştir. TLB, sayfa tablosunun bir alt kümesidir.

### 16.1 Verilenler ve Alanlar

**Verilen:** Sanal adres 20 bit, yani 1 MB. Fiziksel adres 18 bit, yani 256 KB. Sayfa boyutu 1 KB. TLB, 2-way set associative ve toplam 16 girdili.

Kullanılan kısaltmalar: TLBI (TLB index), TLBT (TLB tag), PPN (physical page number), PPO (physical page offset), VPN (virtual page number), VPO (virtual page offset).

1. **VPN ve VPO:** 1 KB sayfa için VPO 10 bittir. VPN $= 20 - 10 = 10$ bittir.
2. **TLBI ve TLBT:** $16/2 = 8$ satır vardır, 0'dan 7'ye kadar. 8 satır için TLBI 3 bittir. TLBT $= 10 - 3 = 7$ bittir.
3. **PPN ve PPO:** PPO 10 bittir. PPN $= 18 - 10 = 8$ bittir.

   **Ders sonrası not:** Hoca "10 bitin 3 biti TLB tag, geriye kalan 7 bit TLB index" dedi; doğrusu TLB index 3 bit, TLB tag 7 bittir. Hoca aynı cümlede doğru değerleri de söyledi.

Her adres için erişilen TLB girdisi, TLB hit ya da miss, page fault olup olmadığı, fiziksel sayfa numarası ve fiziksel adres istenir.

### 16.2 Birinci Adres: 04AA4

20 bit, 5 onaltılık hanedir. Burada alan sınırları 4 bitin katına denk gelmediği için önceki örnekteki gibi onaltılık sayı üzerinden bölünemez, binary'e açmak gerekir.

`04AA4` = `0000 0100 1010 1010 0100`

1. Yüksek 10 bit VPN: `0000010010`. Soldan 2 bit ve iki dörtlü grup olarak yazılır: `00 0001 0010`, yani **012**.
2. Düşük 10 bit VPO: `1010100100`. Gruplanınca `10 1010 0100`, yani **2A4**.
3. VPN'in düşük 3 biti TLBI: `010`, yani **2**.
4. Kalan 7 bit TLBT: `0000010`, yani **02**.
5. TLB'nin 2. satırında tag'i 02 olan ve valid bir girdi vardır: **TLB hit**. TLB valid olduğu için page fault oluşmaz.
6. Girdinin PPN değeri **68**'dir. Hoca değeri bulanık baskıda kontrol ederek okudu.
7. PPN'in arkasına 10 bitlik offset eklenir: `01101000` ve `1010100100`.
8. Sonuç dörtlü yeniden gruplanır: `01 1010 0010 1010 0100`, yani **1A2A4**.

### 16.3 İkinci Adres: 078E6

Hoca bu adresi TLB miss durumunu göstermek için seçti.

`078E6` = `0000 0111 1000 1110 0110`

1. Yüksek 10 bit VPN: `00 0001 1110`, yani **01E**.
2. Düşük 10 bit VPO: `00 1110 0110`, yani **0E6**.
3. TLBI: `110`, yani **6**.
4. TLBT: `0000011`, yani **03**.
5. TLB'nin 6. satırında tag'i 03 olan girdi yoktur: **TLB miss**.
6. VPN sayfa tablosunda aranır. Büyük tablonun 1E satırına gidilir. Satır valid'dir, yani page fault yoktur. PPN **57**'dir.
7. PPN binary yazılır ve arkasına offset eklenir: `01010111` ve `0011100110`.
8. Dörtlü yeniden gruplanır. Sağdan başlanır: 0110 için 6, 1110 için E, 1100 için C, 0101 için 5, kalan `01` için 1. Sonuç **15CE6**'dır.

| Sanal adres | VPN | VPO | TLBI | TLBT | TLB | Page fault | PPN | Fiziksel adres |
|---|---|---|---|---|---|---|---|---|
| 04AA4 | 012 | 2A4 | 2 | 02 | Hit | Hayır | 68 | 1A2A4 |
| 078E6 | 01E | 0E6 | 6 | 03 | Miss | Hayır | 57 | 15CE6 |

### 16.4 Sayfa Tablosunda da Valid 0 Olursa

**Soru:** Burada da bulamazsak aramak için ana belleğe mi gidiyor?

**Cevap:** Satır sayfa tablosunda bulunur, ama valid biti 0 olabilir. Valid 0 ise page fault'tur. Fiziksel bellekte dolu bir gözün üzerine yeniden yazmak, yani replacement yapmak gerekir. Bu sırada dolu gözdeki sayfa önce diske yazılır, diskteki yeni sayfa fiziksel belleğe taşınır. Bu yer değiştirmeye swap denir. Sayfa üzerinde hiç güncelleme yapılmamışsa geri yazmaya gerek kalmaz. Buna dirty bitine bakılarak karar verilir.

> **Ders sonrası not:** Hoca güncellenmemiş sayfa için "main memory'e yazmanıza gerek kalmıyor" dedi; doğrusu diske yazmaya gerek kalmamasıdır. Aynı cevabın başında çıkan sayfanın diske yazıldığı söylenmişti.

Hocaya göre tekniği anladıktan sonra bu örnekler basit ve zevkli bir iştir.

## Dersin Toparlaması

Sanal bellek, önbellek fikrinin ana bellek ile disk arasına taşınmış hâlidir. Birim sayfadır, bulunamama durumu page fault'tur, yerleştirme sayfa tablosu üzerinden fully associative, yazma ise write-back'tir. Sanal adres, VPN ve offset olarak bölünür. VPN sayfa tablosunda satır seçer, offset ise iki adreste de aynı kalır. Fiziksel adres, PPN ile offset'in yan yana yazılmasıyla oluşur. Sayfa tablosu ana bellekte olduğu için dönüşüm pahalıdır. Bu yüzden son kullanılan girdiler TLB adlı özel önbellekte tutulur. Set associative bir TLB'de VPN, tag ve index olarak bölünür ve index'in seçtiği satırdaki bütün setlere bakılır. TLB miss olunca sayfa tablosuna gidilir, orada da valid 0 ise page fault olur. Bellek doluysa FIFO ya da LRU gibi bir politika çıkarılacak sayfayı seçer. Çıkan sayfa dirty ise önce diske yazılır.

Çok seviyeli sayfa tabloları, parçalanma, erişim hakları ve diğer yerine koyma algoritmaları İşletim Sistemleri dersinde ayrıntılı işlenecektir. Bu derste öğrenilen donanım altyapısı o konuların üzerine oturacağı temeldir.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.